

Directrices mínimas para la elección del tema del Proyecto Semestral

Asignatura: Desarrollo Fullstack I

Las siguientes directrices establecen los **requisitos mínimos que debe cumplir un proyecto semestral para ser aceptado** dentro de la asignatura. Estos lineamientos buscan asegurar un **nivel de complejidad acorde al tercer semestre de la carrera**, garantizando que todos los proyectos desarrollados en las distintas sedes mantengan un estándar académico equivalente.

Es importante considerar que **los requisitos aquí indicados representan el mínimo exigido**, por lo que los equipos pueden desarrollar proyectos con mayor complejidad o cantidad de componentes si lo consideran necesario.

Contexto del proyecto

El proyecto semestral consiste en el diseño e implementación de un sistema basado en una arquitectura de microservicios.

- **Equipos:** Mínimo 2, máximo 3 integrantes.
- **Evaluaciones:** * Evaluación 2: Proyecto funcional + Defensa oral.
 - Evaluación 3: Proyecto optimizado (Pruebas, Documentación y Despliegue) + Defensa oral.
- **Requisito de Complejidad:** El tema debe representar un problema real, evitando aplicaciones simples como listas de tareas o agendas.

El tema del proyecto debe cumplir las siguientes condiciones:

- Representar **un problema real o una necesidad concreta de un sistema de información**.
- Corresponder a **un sistema completo de gestión o servicio digital**, evitando propuestas excesivamente simples o meramente académicas.
- Tener **un nivel de complejidad acorde a estudiantes de tercer semestre**, quienes ya dominan los fundamentos de programación y bases de datos.
- Incluir **múltiples módulos funcionales y componentes backend** que justifiquen el uso de arquitectura de microservicios.

Arquitectura del sistema

El sistema deberá desarrollarse **obligatoriamente utilizando arquitectura de microservicios con Spring Boot**.

El proyecto debe considerar como mínimo:

- **10 microservicios independientes.**
- Cada microservicio debe:
 - Tener **una responsabilidad clara dentro del sistema.**
 - Manejar **su propia lógica de negocio.**
 - Exponer **API REST para su consumo.**
 - Contar con **sus propios endpoints funcionales.**
- Los microservicios deben estar **completamente desacoplados entre sí.**
- La cantidad indicada corresponde al **mínimo exigido**, pudiendo implementarse más microservicios si el proyecto lo requiere.

Comunicación entre microservicios

El sistema debe demostrar **interacción real entre los distintos microservicios**, incluyendo:

- Consulta de datos entre servicios.
- Validación de información entre microservicios.
- Flujo de datos entre distintos módulos del sistema.
- Interacción mediante mecanismos de comunicación apropiados, tales como:
 - WebClient
 - u otros mecanismos equivalentes de comunicación entre servicios.

El sistema debe contar además con un **API Gateway y un Service Register como Eureka**, o cualquier otro mecanismo que ustedes consideren adecuado.

Persistencia de datos

Cada microservicio debe contar con **su propia base de datos independiente.**

Se deben cumplir las siguientes condiciones:

- No se permite **compartir tablas entre microservicios.**
- No se deben duplicar estructuras de datos entre servicios.
- Las bases de datos deben estar **correctamente normalizadas.**
- Debe existir un **modelo relacional validado mediante diagramas de base de datos.**

Los motores de base de datos permitidos son:

- **MySQL (recomendado)**
- **Oracle**

Los entornos sugeridos para su implementación incluyen:

- XAMPP
- Laragon

El proyecto debe evidenciar:

- entidades correctamente modeladas
- relaciones entre entidades
- integridad de los datos
- validaciones y reglas de negocio aplicadas a nivel de aplicación.

Roles de usuario

El sistema debe contemplar **entre 2 y 3 roles de usuario diferenciados**.

Cada rol debe:

- tener **funcionalidades distintas dentro del sistema**
- manejar **permisos y privilegios diferentes**
- interactuar con **distintos módulos del sistema**

Ejemplos de roles:

- Administrador
- Usuario cliente
- Operador del sistema

Funcionalidades del sistema

El sistema debe incluir funcionalidades suficientes para justificar su arquitectura.

Como mínimo debe considerar:

- múltiples operaciones **CRUD**
- operaciones **personalizadas adicionales a los CRUD básicos**
- **validaciones de datos**
- **reglas de negocio**
- **consultas a bases de datos**
- **flujo de información entre microservicios**

Las APIs desarrolladas deben permitir **gestionar completamente los procesos del sistema**.

Seguridad básica del sistema

El sistema debe implementar **mecanismos básicos de seguridad**, tales como:

- encriptación de contraseñas antes de almacenarlas
- autenticación de usuarios
- generación de tokens de sesión en el proceso de login
- validación de accesos según roles de usuario.
- Cualquier otro que garantice la seguridad, protección y acceso de datos e información.

Control de versiones y repositorio

Todos los proyectos deben utilizar obligatoriamente:

GitHub como repositorio de código.

El repositorio debe permitir evidenciar:

- avance progresivo del desarrollo
- participación de todos los integrantes del equipo
- historial de cambios del proyecto.

Se recomienda trabajar con:

- ramas de desarrollo
- commits frecuentes
- estructura organizada del proyecto.

Documentación, pruebas y despliegue (Experiencia de Aprendizaje 3)

Durante el desarrollo del proyecto se deberán incorporar elementos adicionales tales como:

- documentación técnica del sistema
- pruebas unitarias
- despliegue de los microservicios.

El despliegue puede realizarse mediante:

- entorno local
- herramientas de exposición de puertos
- servicios gratuitos de despliegue.

El sistema debe quedar disponible mediante **una URL que permita consumir las APIs del proyecto**.

ejemplos de proyectos aceptables

Los siguientes ejemplos ilustran el tipo de sistemas que cumplen con el nivel de complejidad esperado:

- Sistema de gestión de reservas de hoteles
- Plataforma de gestión de pedidos para restaurantes
- Sistema de gestión de bibliotecas
- Plataforma de gestión de ventas online
- Sistema de gestión de inventario con logística
- Marketplace de servicios o productos
- Plataforma de gestión de reservas de servicios

Ejemplos de proyectos no aceptables

No se aceptarán proyectos excesivamente simples, tales como:

- listas básicas de tareas
- agendas simples
- CRUD simples sin lógica de negocio
- sistemas que no requieran interacción entre microservicios
- aplicaciones con una única base de datos compartida.

Herramientas y Metodologías Sugeridas

Herramienta / Técnica	¿Para qué sirve? (Explicación Sencilla)
I. Arquitectura	
Spring Boot	El framework principal que se usará para crear cada uno de los microservicios de forma rápida y estandarizada.
Microservicios Desacoplados	Técnica de diseño donde cada módulo (ej: Ventas, Inventario) es independiente, maneja su propia base de datos y no depende directamente de los otros para funcionar.
II. Base de Datos	
Base de Datos por Servicio	Patrón obligatorio donde cada microservicio tiene su propia base de datos (MySQL, PostgreSQL u Oracle), prohibiéndose compartir tablas directamente.
Spring Data JPA	Biblioteca que facilita la interacción entre el código Java y la base de datos sin escribir SQL complejo manualmente.
Diagrama Entidad-Relación (DER)	Técnica de modelado para planificar y validar cómo se estructuran los datos antes de crear las tablas reales.
III. Documentación	
SpringDoc OpenAPI / Swagger	Genera automáticamente una página web interactiva para visualizar y probar las APIs (endpoints) directamente desde el navegador.

Anotaciones de Swagger	Etiquetas especiales en el código (ej: @Operation) que añaden descripciones amigables a la documentación automática.
Markdown (README.md)	Archivo de texto guía en el repositorio que explica de qué trata el proyecto, cómo instalarlo y cómo ejecutarlo.
IV. Pruebas (Calidad)	
JUnit 5	El framework estándar para escribir y ejecutar pruebas automáticas que verifican si la lógica del código funciona correctamente.
Mockito	Biblioteca para simular comportamientos de objetos complejos (como bases de datos) y probar tus servicios de forma aislada y segura.
Spring Boot Validation	Uso de anotaciones (ej: @NotNull) en tus datos (DTOs) para asegurar que la información que recibes sea correcta antes de procesarla.

V. Comunicación	
Feign Client	Biblioteca para que los microservicios se comuniquen entre sí de forma sincrónica (esperando respuesta inmediata) mediante llamadas HTTP sencillas.
Apache Kafka	Plataforma de mensajería para comunicación asincrónica (sin esperar respuesta), usada para sincronizar datos mediante eventos entre microservicios.
Eureka (Service Discovery)	Un directorio dinámico donde cada microservicio se registra al iniciar, permitiendo que otros lo localicen automáticamente por su nombre.
API Gateway	El punto de entrada único para todas las solicitudes externas, que redirige el tráfico al microservicio correspondiente.
VI. Seguridad	
Spring Security	Framework completo para gestionar la autenticación (quién eres) y autorización (qué puedes hacer) en todo el sistema.
BCrypt	Algoritmo de cifrado obligatorio para proteger y guardar contraseñas de forma segura en la base de datos.
JSON Web Token (JWT)	Biblioteca para generar tokens de sesión seguros y sin estado, permitiendo que el usuario permanezca logueado sin usar sesiones de servidor.
Control de Acceso (RBAC)	Técnica para restringir el acceso a ciertas partes de la API basándose en los roles del usuario (ej: Administrador vs Cliente).
VII. Despliegue (DevOps)	
Maven / Gradle	Herramientas para gestionar todas las bibliotecas del proyecto y empaquetar el código en un archivo .jar ejecutable.
Docker / Docker Compose	Tecnología para "contenerizar" la aplicación y base de datos, asegurando que funcionen igual en cualquier PC, e iniciarlas con un solo comando.
Ngrok / Localtunnel	Herramientas que crean un túnel seguro para exponer el servidor local a internet, dándote una URL pública temporal para pruebas externas.
Railway / Render / Koyeb	Servicios en la nube gratuitos (o con capa gratuita) para alojar el proyecto en producción de forma permanente y con una URL fija.
VIII. Gestión de Código	
GitHub	Plataforma obligatoria para almacenar el código, controlar versiones, trabajar en equipo y evidenciar el progreso.
Branches (Ramas) / Commits	Prácticas de Git para organizar el desarrollo de nuevas funciones y registrar un historial claro de los cambios del proyecto.